1. CSS Transitions (Hover Effects)

Definition:
CSS transitions allow changes in CSS properties to occur smoothly over a specified duration rather than instantly.

Syntax:

```
selector {
  transition: property duration timing-function delay;
}
```

Important Properties:

- transition-property: Which CSS property to animate (e.g., color, background-color).
- transition-duration: How long the transition takes (s or ms).
- transition-timing-function: How the speed curve behaves (ease, linear, ease-in, ease-out, etc.).
- transition-delay: How long to wait before the transition starts.

Example:

```
.button {
  background-color: blue;
  color: white;
  transition: background-color 0.5s ease-in-out;
}

.button:hover {
  background-color: red;
}
```

➡️ When the user hovers over the .button, it smoothly changes from blue to red in 0.5 seconds.

2. CSS Animations

Definition:
CSS animations allow elements to change styles over time using keyframes.

Syntax Overview:

```
@keyframes animationName {
  0%   { property: value; }
  50%  { property: value; }
  100% { property: value; }
}

.selector {
  animation-name: animationName;
  animation-duration: 2s;
  animation-delay: 1s;
  animation-iteration-count: infinite;
  animation-direction: alternate;
  animation-timing-function: ease-in-out;
}
```

Key Animation Properties:

Property	Description
@keyframes	Defines the animation steps
animation-name	Name of the keyframe
animation-duration	How long the animation takes
animation-delay	Delay before the animation starts
animation-iteration-count	Number of times animation should repeat (e.g., 1, infinite)
animation-direction	Direction of animation (normal, reverse, alternate)
animation-timing-function	Speed curve (ease, linear, etc.)
animation-fill-mode	How styles are applied before/after animation (forwards, backwards)

Example:

```
@keyframes slide {
  from { transform: translateX(0); }
  to   { transform: translateX(200px); }
}

.box {
  width: 100px;
  height: 100px;
  background: green;
```

```
  animation: slide 2s ease-in-out infinite alternate;
}
```

The green box will move 200px to the right and come back in a smooth loop.

3. CSS Transform

Definition:
The transform property lets you rotate, scale, skew, or move elements in 2D or 3D space.

Common Transform Functions:

Function	Description	Example
translate(x, y)	Moves element along X and Y axis	transform: translate(50px, 20px);
rotate(deg)	Rotates the element clockwise/counterclockwise	transform: rotate(45deg);
scale(x, y)	Scales element in width and height	transform: scale(1.5, 2);
skew(x, y)	Skews element by angles along X and Y	transform: skew(30deg, 0);

Example:

```
.card:hover {
  transform: scale(1.1) rotate(5deg);
  transition: transform 0.3s ease;
}
```

On hover, the .card will grow slightly and rotate a bit.

Bonus: Combining Transitions + Transforms + Animations

```
@keyframes bounce {
  0%   { transform: translateY(0); }
  50%  { transform: translateY(-30px); }
  100% { transform: translateY(0); }
}

.ball {
  width: 50px;
  height: 50px;
  background: red;
  border-radius: 50%;
  animation: bounce 1s infinite;
}
```

Summary Table

Feature	Use	Syntax Example
Transition	Smooth property changes	transition: all 0.5s ease;
Animation	Repeated/complex timed animations	@keyframes + animation properties
Transform	Move/scale/skew/rotate elements	transform: rotate(45deg);

Responsive Design & Media Queries

1. What is Responsive Design?

Definition:
Responsive Design is a web design approach that ensures a website's layout and content adapt to various screen sizes and devices (desktop, tablet, mobile).

Goals:

- Provide an optimal user experience on all devices.
- Eliminate the need for zooming or horizontal scrolling.
- Make content flexible and fluid.

How?

- Flexible layouts using relative units (%, em, rem, etc.)
- CSS Media Queries to apply different styles on different screen widths.
- Use of flexible images and containers.

2. Viewport and Media Queries

Viewport

Definition:
The visible area of a web page on a device screen.

Important Meta Tag:

<meta name="viewport" content="width=device-width, initial-scale=1.0">

- width=device-width: Sets the viewport width equal to the device width.

- initial-scale=1.0: Sets initial zoom level.

Without this tag, your layout may appear zoomed out or broken on mobile devices.

Media Queries

Definition:
Media queries are CSS techniques that allow applying styles conditionally based on screen size or device characteristics.

Syntax:

```
@media (condition) {
  /* CSS rules */
}
```

Examples:

```
/* Mobile (up to 600px) */
@media (max-width: 600px) {
  body {
    background-color: lightblue;
  }
}

/* Tablets (601px to 900px) */
@media (min-width: 601px) and (max-width: 900px) {
  body {
    background-color: orange;
  }
```

```
}

/* Desktops (above 900px) */
@media (min-width: 901px) {
  body {
    background-color: lightgreen;
  }
}
```

This technique lets you build device-specific layouts for mobile, tablet, and desktop.

3. Mobile-First Design

Definition:
Mobile-First Design is a strategy where you design and write CSS styles for small screens **first**, and then scale up for larger devices using media queries.

Approach:

1. Base styles for mobile (default styles).
2. Use min-width media queries to apply changes for larger screens.

Example:

```
/* Base (mobile-first) */
.container {
  font-size: 14px;
  padding: 10px;
```

```
}

/* Tablet and up */
@media (min-width: 768px) {
  .container {
    font-size: 16px;
    padding: 20px;
  }
}

/* Desktop and up */
@media (min-width: 1024px) {
  .container {
    font-size: 18px;
    padding: 30px;
  }
}
```

4. Units: %, em, rem, vh, vw

Unit	Meaning	Use Case
%	Relative to parent element	Widths, paddings
em	Relative to parent font-size	Typography, spacing
rem	Relative to root (html) font-size	Global sizing consistency
vh	1% of viewport height	Fullscreen sections
vw	1% of viewport width	Responsive width

Example:

```
.hero {
  width: 80%;
  font-size: 2rem;
  height: 100vh;
  padding: 2em;
}
```

5. Hiding/Showing Elements on Different Screens

Use display, visibility, or custom classes with media queries to hide/show elements.

Example with Media Query:

```
/* Hide on mobile */
@media (max-width: 600px) {
  .desktop-only {
    display: none;
  }
}

/* Show only on mobile */
@media (min-width: 0px) and (max-width: 600px) {
  .mobile-only {
    display: block;
  }
}
```

Summary Table

Feature	Description	Example
Viewport	Controls how content is scaled on different devices	<meta name="viewport" ...>
Media Queries	Apply CSS based on screen width/device	@media (max-width: 768px)
Mobile-First	Design for small screens first	Default → min-width queries
Responsive Units	Use flexible sizing	%, em, rem, vh, vw
Show/Hide	Conditional display of elements	.desktop-only { display: none; }

CSS Variables and Best Practices

1. Custom Properties (CSS Variables)

Definition:
CSS Variables (also called Custom Properties) allow you to store values (like colors, fonts, spacing) in a reusable way.

Syntax:

--variable-name: value;

Usage:

```
:root {
  --main-color: #3498db;
  --secondary-color: #2ecc71;
  --font-stack: 'Segoe UI', sans-serif;
  --spacing: 16px;
}

body {
  color: var(--main-color);
  font-family: var(--font-stack);
  padding: var(--spacing);
}
```

:root is a pseudo-class that matches the highest-level element (usually <html>) and is commonly used to define global CSS variables.

Example:

```
:root {
  --button-bg: #e91e63;
  --button-text: white;
}

button {
  background-color: var(--button-bg);
  color: var(--button-text);
}
```

Easy to change theme colors in one place!

2. Using Variables for Theme Colors, Fonts, Spacing

Using CSS variables makes theming your site much easier.

Theme Example:

```
:root {
  --bg-color: #ffffff;
  --text-color: #333;
  --primary-font: 'Poppins', sans-serif;
  --section-padding: 40px;
}

.dark-theme {
  --bg-color: #1e1e1e;
  --text-color: #f5f5f5;
```

```
}

body {
  background-color: var(--bg-color);
  color: var(--text-color);
  font-family: var(--primary-font);
  padding: var(--section-padding);
}
```

This allows switching between themes (e.g., light/dark mode) easily just by toggling a class.

3. Organizing CSS Code – Best Practices

Keeping CSS clean, consistent, and modular improves maintainability.

Best Practices:

Practice	Description
Use :root for global variables	Define theme-wide values once in :root
Use descriptive names	Like --primary-color, --heading-font, --base-spacing
Group related variables	Colors, Fonts, Spacing, Breakpoints separately
Avoid hardcoded values	Replace repeated values with variables
Use modular CSS structure	Separate files for layout, components, themes

Practice	Description
Follow consistent naming	Use kebab-case: --section-padding, --card-bg
Use comments to separate sections	/* Theme Colors */, /* Typography */ etc.

Organized CSS Variables Example:

```
:root {
  /* Theme Colors */
  --primary-color: #4CAF50;
  --secondary-color: #FFC107;
  --text-color: #333;

  /* Typography */
  --font-main: 'Open Sans', sans-serif;
  --font-size-base: 16px;
  --font-size-heading: 24px;

  /* Spacing */
  --spacing-sm: 8px;
  --spacing-md: 16px;
  --spacing-lg: 32px;

  /* Layout */
  --max-width: 1200px;
```

}

Summary Table

Feature	Syntax	Example
Define Variable	--var-name: value;	--main-color: red;
Use Variable	var(--var-name)	color: var(--main-color);
Global Variables	Use :root	:root { --font: 'Poppins'; }
Theme Switching	Override variables in a class	. dark-theme {--bg: #000; }
Maintainable Code	Group variables & use comments	/* Typography */ --font-size: 16px;

Introduction to Framework (Bootstrap)

1. What is a CSS Framework?

Definition:
A **CSS framework** is a pre-prepared library that provides a set of common CSS classes and JavaScript components, helping you build responsive and styled web pages faster without writing CSS from scratch.

2. What is Bootstrap?

Bootstrap is the **most popular HTML, CSS, and JavaScript framework** used for developing **responsive and mobile-first websites**.

Created by:

- Twitter developers
- Initially released in 2011

Current Version:

- As of now, the latest is **Bootstrap 5** (no jQuery dependency)

3. Key Features of Bootstrap:

Feature	Description
Responsive Grid System	12-column layout that adapts to screen sizes
Predefined CSS classes	Ready-made styles for buttons, text, cards, navbars
Mobile-First	Designed to work on small devices first

Feature	Description
Components	Ready-to-use UI components like modals, alerts, navbars
Utility Classes	Classes for spacing, colors, typography, etc.
JavaScript Plugins	Built-in functionality like carousels, tooltips, dropdowns

4. How to Use Bootstrap

Include Bootstrap in HTML (via CDN):

<! -- Bootstrap 5 CSS -->

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">

<! -- Bootstrap 5 JS (Optional for components like modals, dropdowns) -->

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>

5. Basic Structure Example

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

```
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Bootstrap Example</title>
 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>

 <div class="container">
  <h1 class="text-primary">Hello, Bootstrap! </h1>
  <button class="btn btn-success">Click Me</button>
 </div>

</body>
</html>
```

6. Bootstrap Grid System (Responsive Layouts)

Bootstrap uses a 12-column responsive grid:

```
<div class="container">
 <div class="row">
  <div class="col-6">Column 1</div>
  <div class="col-6">Column 2</div>
 </div>
</div>
```

- col-6 = 6 out of 12 columns
- Adjusts automatically on screen sizes

7. Popular Bootstrap Components

Component	Class Example
Buttons	btn, btn-primary, btn-danger
Alerts	alert alert-success, alert alert-warning
Navbar	navbar, navbar-expand-lg, navbar-light
Cards	card, card-body, card-title
Modal	modal, modal-dialog, modal-body
Carousel	carousel, carousel-item

Summary Table

Topic	Description
Bootstrap	Open-source CSS + JS framework
Use	Build responsive, mobile-first websites
Includes	Grid system, components, utilities

Topic	Description
Setup	Add CSS/JS via CDN or download
Benefits	Saves time, ensures consistency, mobile-ready